

Proof-Carrying Verification Conditions for SPARK: An Experience Report

Tony Gair

9 August 2026

Proof-Carrying Verification Conditions for SPARK: An Experience Report

Tony Gair · 9 August 2026 Preprint. The Dark Factory Ltd, South Shields, United Kingdom.

Author: Tony Gair — ORCID 0009-0006-6279-3677. Licensed CC BY 4.0.

Abstract

Industrial deployments of SPARK/Ada rely on `gnatprove` and its SMT back ends to discharge verification conditions (VCs). The verdict, however, is a claim by the verifier: a downstream consumer of a verified component must either trust the producer’s toolchain or re-run the proof themselves, which is expensive. We report an experiment attaching **independently checkable proof certificates** to the SMT step of SPARK verification. Using the `cvc5` binary already shipped inside the SPARK toolchain, we emit Cooperating Proof Calculus (CPC) certificates and validate them with **Ethos**, a checker sharing no code with the producing solver. On two industrial SPARK components we obtain **28 of 31 verification conditions independently checked**, including **13 of 13** on an integer-arithmetic taxation component, at a mean checking cost of approximately **0.03 s per certificate** — three orders of magnitude below re-proving. The three failures are a single, precisely located rewrite. We report the method, the measurements, the version-compatibility hazard that makes the pipeline brittle, and the limits of what the resulting artifact establishes.

1. Introduction

A formally verified software component is normally distributed with a *claim*: the producer states that proof obligations were discharged. A recipient who wishes to rely on that claim has two options, and both are poor. They may trust the producer, their toolchain and their solver configuration — which is a social rather than a mathematical guarantee. Or they may re-run the verification, which requires the same toolchain, the same versions, and minutes to hours of compute per component.

This is the gap Proof-Carrying Code identified nearly thirty years ago [1, 2]: the producer should spend the effort, and the consumer should check a *certificate* cheaply. PCC did not become the norm, and the usual explanation is supply-side: producing proofs required a human expert per artifact.

Two things have changed. Automated verification of industrial code is now routine in some domains; and SMT solvers have acquired fine-grained, machine-checkable proof output. We ask a narrow, empirical question:

Can a SPARK component, verified by the standard toolchain, ship a certificate that a third party validates cheaply, without trusting the producer?

We answer partially yes, quantify it, and state precisely which part of the chain remains un-certified.

2. Background

SPARK and gnatprove. SPARK is a verifiable subset of Ada. **gnatprove** translates annotated source into the Why3 intermediate language, generates VCs by a weakest-precondition calculus, and discharges them with SMT solvers (in the configuration studied here, **cvc5**).

CPC and Ethos. The Cooperating Proof Calculus is a proof system covering the inferences **cvc5** uses in its mainstream theories, expressed in the **Eunoia** logical framework. **Ethos** is an independent checker for Eunoia proofs. Ethos does not implement a fixed calculus; the calculus is supplied declaratively, which makes the checker small relative to the solver.

Trust base. A certificate for the SMT step reduces reliance on the solver — historically the largest and least auditable component — to reliance on a checker plus a calculus definition.

3. Method

For each component:

1. Run **gnatprove -P <project>.gpr --level=2 --debug**, which preserves the generated VCs as SMT-LIB files under **obj/gnatprove/**.
2. For each VC, run **cvc5** with **--dump-proofs --proof-format-mode=cpc --proof-granularity=dsl-rewrite**.
3. Strip the leading **unsat** line, and any trailing (**error ...**) emitted *after* the refutation (see §6.2).
4. Validate with Ethos against the CPC signature set matching the solver version.

Proof granularity is decisive and, we believe, under-documented. At default granularity the certificates are not hole-free; the same obligations produce complete certificates at **dsl-rewrite**:

--proof-granularity	unjustified steps (toy QF_LIA instance)
default	17
macro	11
theory-rewrite	17
dsl-rewrite	0

4. Subjects

Two SPARK components taken from an industrial corpus of proven cores, chosen to contrast theories rather than to be representative:

- **C1 — a nonce-discipline component.** Contracts quantified over sequences; makes a specific unsafe state unconstructible. Quantifier-heavy.

- **C2 — a finance-cost relief component.** Taxation arithmetic; amounts are integer minor units (`Long_Long_Integer`), no floating point, no fixed-point types. `gnatprove` reports 9 checks, 0 unproved, 0 justified.

5. Results

	C1 (quantified)	C2 (integer arithmetic)
Verification conditions	18	13
Hole-free certificates	15	13
Independently checked (correct)	15 (83%)	13 (100%)
Mean check time	~0.03 s	~0.03 s

A 626-step certificate for one postcondition of C1 was validated in 32 ms.

The three failures are one defect. All occur at a single source location in C1 (a postcondition and two of its conjuncts), and each certificate contains **exactly one** unjustified step. In each case the unjustified obligation is a **quantifier rewrite**, relating a universally quantified formula to a disjunctive form. The blocker is therefore not “SPARK proofs are uncertifiable” but a single rewrite that the calculus does not justify at this granularity.

Theory matters more than size. The integer-arithmetic component certified completely; the quantified one did not. If this generalises, the components most amenable to certification are exactly those handling money, allocation, bounds and framing — a commercially significant class.

6. Practical hazards

6.1 Version coupling. Solver, signature set and checker must be contemporaneous. A current checker rejects older signatures (a command removed from the framework); current signatures reject older proofs (a step fails to check). Three combinations failed before one worked. Any deployment must ship the three pinned together, because the failure presents as *proof invalid* rather than *versions mismatched* — a dangerous confusion in a security context.

6.2 A benign error that looks like failure. Why3 appends a `get-info` command to its VC files. `cvc5` emits an error for this *after* producing a complete refutation. Left in place, the trailing error causes the checker to reject an otherwise valid certificate.

7. Limitations and threats to validity

We state these plainly because the artifact is easy to over-read.

1. **The certificate covers the SMT step only.** It establishes that the verification conditions are valid. It does **not** establish that they faithfully represent the source program: the Ada→Why3 translation and VC generation remain trusted and uncertified here. A valid proof of the wrong obligations remains worthless. Work on formalising Why3’s core exists and is the obvious complement.
2. **The checker is not itself verified.** It is small and independent, but it is in the trust base.
3. **Two components.** No claim of generality; the theory contrast is suggestive, not established.
4. **Floating-point excluded.** Hole-free CPC generation does not currently extend to FP arithmetic; components using it are out of scope.

5. **Our related-work survey (§7a) was conducted rapidly and is not systematic.** We found no prior report of a SPARK component shipping an independently-checked SMT certificate, and we invite correction from readers who know otherwise.

7a. Related work

Proof-carrying code. The idea originates with Necula and Lee (POPL 1997): a host determines that code from an untrusted source is safe to run by checking an accompanying certificate. The line developed through Typed Assembly Language (Morrisett, Walker et al., 1999), certifying compilers for Java (Colby, Lee, Necula et al., 2000), and **Foundational PCC** (Appel, Princeton, 1999–2005), which shrank the trusted base by deriving proofs from the foundations of logic with no type-specific axioms — at the cost of proofs that were, in the project’s own assessment, very much harder to construct. Later work extended the approach to self-modifying code (Cai, Shao, Vaynberg, 2007) and to low-level programs with preemptive threads and interrupts (Feng, Shao, Guo, Dong, 2009); Abstraction-Carrying Code (Albert, Puebla, Hermenegildo) addressed certificate size.

The standard account of why PCC did not reach practice is supply-side: **generating the proofs was the roadblock**, requiring expert effort per artifact. Our contribution is not a new idea but an observation about that constraint: where verification of industrial code is already automated, the certificate is close to free, and the historical objection does not apply.

The idea is currently being revisited for machine-generated code — see *Proof-Carrying Code Completions* (Kamran, Stanford et al., ASEW 2024).

Certifying the verifier. VST and CakeML take the foundational route, implementing the toolchain inside a proof assistant. Closest to our unclosed link is **Cohen and Johnson-Freyd, “A Formalization of Core Why3 in Coq” (POPL 2024)**, which gives a Coq semantics for Why3’s logic fragment, a correct-by-construction proof system, and soundness proofs for two Why3 transformations. That work addresses precisely the layer our certificates do not reach.

SMT proof formats and checkers. Alethe (Schurr, Fleury, Barbosa, Fontaine, EPTCS 336, 2021) with the Carcara checker (TACAS 2023); LFSC; and the calculus used here — the **Cooperating Proof Calculus** (CAV 2026) in the **Eunoia** framework, checked by **Ethos** (IJCAR 2026). Carcara’s authors note that the checker itself is in the trusted base, a caveat that applies equally to Ethos and to us.

SPARK in practice. AdaCore’s *Focused Certification of an Industrial Compilation and Static Verification Toolchain* documents the qualification approach for the toolchain itself. Contemporaneously with this work, **Philipp, “The Prover Is the Judge: Verified Security Software from AI Coding Agents in Ada/SPARK” (arXiv 2607.14340, July 2026)** reports bare-metal security software — TLS 1.3, IKEv2, X.509, post-quantum cryptography — produced by AI agents with **49,280 obligations discharged by gnatprove**. That work is far larger than ours in scale and addresses production; it does not attach independently checkable certificates to the resulting artifacts, which is the gap we address here. The two are complementary: the more verification is automated, the more the question of *who checks the checker’s verdict* matters.

8. Conclusion

The solver step of SPARK verification can be made independently checkable today, with the toolchain already in use, at negligible checking cost — completely for integer-arithmetic components, and for the large majority of a quantified one. What remains is not the solver but the front

end: certifying that the obligations proved are the obligations meant. That is the substantial open problem, and it is where the remaining trust in a “verified” component lives.

Artifact

A self-verifying bundle accompanies this report: sources, proof summary, all certificates, the matched signature set, and a script that re-checks every obligation on the reader’s machine.

References

Proof-carrying code

- [1] G. C. Necula. *Proof-Carrying Code*. In Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’97), Paris, pp. 106–119, 1997. DOI: 10.1145/263699.263712. (Recipient of the Most Influential POPL 1997 Paper Award, 2007.)
- [2] G. C. Necula and P. Lee. *Safe Kernel Extensions Without Run-Time Checking*. In Proc. 2nd USENIX Symposium on Operating Systems Design and Implementation (OSDI ’96), 1996. — the precursor in which the approach first appears.
- [3] A. W. Appel. *Foundational Proof-Carrying Code*. In Proc. 16th Annual IEEE Symposium on Logic in Computer Science (LICS ’01), 2001. — reduces the trusted base to the foundations of logic, at a substantial cost in proof construction.
- [4] G. Morrisett, D. Walker, K. Crary and N. Glew. *From System F to Typed Assembly Language*. ACM Transactions on Programming Languages and Systems 21(3), 1999.

Certifying the verifier

- [5] J. M. Cohen and P. Johnson-Freyd. *A Formalization of Core Why3 in Coq*. Proceedings of the ACM on Programming Languages 8(POPL), pp. 1789–1818, January 2024. DOI: 10.1145/3632902. — formal Coq semantics for Why3’s logic fragment, a correct-by-construction proof system, and soundness proofs for two of Why3’s transformations. Addresses precisely the layer this report’s certificates do not reach.
- [6] *A Framework for Proof-Carrying Logical Transformations*. arXiv:2107.02352.

SMT proof formats and checkers

- [7] H.-J. Schurr, M. Fleury, H. Barbosa and P. Fontaine. *Alethe: Towards a Generic SMT Proof Format*. Proceedings of the 7th Workshop on Proof eXchange for Theorem Proving (PxTP), EPTCS 336, 2021.
- [8] B. Andreotti, H. Lachnitt and H. Barbosa. *Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format*. In Proc. TACAS 2023. — note the authors’ own caveat that the checker is itself in the trusted base; the same applies to Ethos and to this work.
- [9] *Cooperating Proof Calculus*. CAV 2026. — 585 proof rules covering the inferences cvc5 uses in its mainstream theories, expressed in the Eunoia framework.
- [10] *Ethos: An Efficient Proof Checker for the Eunoia Logical Framework*. IJCAR 2026. Implementation: github.com/cvc5/ethos.
- [11] cvc5 documentation, CPC proof output. cvc5.github.io/docs — states that proofs are produced without holes for SMT-LIB benchmarks other than those involving floating-point arithmetic.

SPARK and verified software in practice

[12] AdaCore. *Focused Certification of an Industrial Compilation and Static Verification Toolchain*. — qualification approach for the SPARK toolchain itself.

[13] J. Philipp. *The Prover Is the Judge: Verified Security Software from AI Coding Agents in Ada/SPARK*. arXiv:2607.14340, July 2026. — bare-metal security software (TLS 1.3, IKEv2, X.509, post-quantum) produced by AI agents, with 49,280 obligations discharged by gnatprove. Contemporaneous, far larger in scale, and complementary: it does not attach independently checkable certificates to the resulting artifacts.

[14] *Proof-Carrying Code Completions*. ASEW 2024. — the PCC idea revisited for machine-generated code.

Bibliographic note. Entries [1], [2], [5], [7] and [13] were verified against primary sources. Entries [3], [4], [6], [8]–[12] and [14] are cited from secondary sources and their page numbers, author lists and venues should be confirmed against the publishers’ records before any submission beyond a preprint deposit.

Tooling and AI-use statement

The experiments reported here were conducted, and this report drafted, in an AI-assisted working session using Anthropic’s Claude (Claude Code, Opus 5). The author is neurodivergent and works remotely; AI assistance is used as an accessibility accommodation, in the way another researcher might work with a scribe or an amanuensis.

Per prevailing venue policy an AI system is not an author of this work and is not credited as one. The author takes full responsibility for its contents.

Two remarks are due about what that assistance can and cannot affect. The measurements in §5 were produced by named tools at pinned versions — **gnatprove**, **cvc5 1.2.1**, **Ethos at commit 45e29dd** — and are reproducible from the accompanying artifact by any reader, independently of how the session that produced them was conducted. Conversely the judgements in §7 — what the certificates do *not* establish, and where the trust actually remains — are the parts a reader should scrutinise, and they are stated as narrowly as the author could manage precisely because AI assistance makes overclaiming fluent and easy.

For the record of who did what: the version-coupling diagnosis in §6.1 was proposed by the tooling after three failed attempts. The decision to report the three failures in §5 rather than omit them, and the scope of every claim in §7, were the author’s.